Clean Architecture

dependencies point inward

Watch the 4-min video

In one line: clean architecture arranges code in rings (entities, use cases, interface adapters, frameworks) under one rule: source-code dependencies point only inward, so business logic never depends on the web framework, the ORM or the database.

1 Database models everywhere

In a typical web project the ORM models show up in controllers, templates and business rules. Changing the framework or the database means rewriting everything, and testing a rule needs a running database. Robert C. Martin's **clean architecture** (2012) fixes this with one strict rule about which code may depend on which.

2 The rings

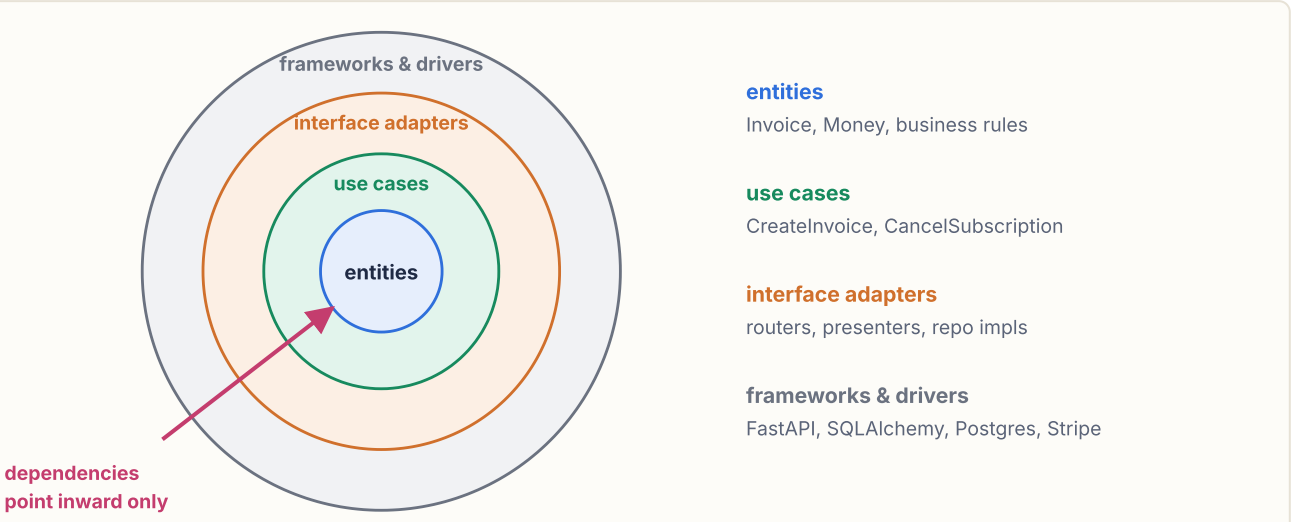


Figure 1. Four rings, one rule. Outer rings know about inner rings, never the reverse.

- **Entities:** core business objects and rules that would exist even without software.
- **Use cases:** application actions that orchestrate entities: *place order*, *cancel subscription*.
- **Interface adapters:** controllers, presenters, repository implementations: convert between use-case data and external formats.
- **Frameworks and drivers:** the web framework, ORM, database, message broker, external APIs.

3 The dependency rule

Source-code dependencies point **only inward**. An entity never imports the web framework; a use case never imports the ORM. Data crossing a boundary is passed as simple structures that belong to the inner ring (a command object, a DTO), not as ORM rows or HTTP requests.

How does a use case save without importing the database? The same trick as hexagonal architecture (lesson 31): the use case defines a `InvoiceRepository` interface, an outer ring implements it, and the

composition root wires the real implementation in at start-up. Control flows outward; dependencies point inward.

```
# adapters/http.py  (outer ring)
@router.post('/invoices')
def create(req: CreateInvoiceRequest):
    return CreateInvoice(repo).run(req.to_cmd())

# usecases/create_invoice.py  (inner ring: no FastAPI, no SQLAlchemy)
class CreateInvoice:
    def __init__(self, repo: InvoiceRepository): self.repo = repo
    def run(self, cmd): invoice = Invoice.create(cmd.customer_id, cmd.lines)
                        self.repo.add(invoice); return invoice.id

# infrastructure/sql_invoice_repo.py implements InvoiceRepository
```

Figure 2. Thin router, use case depending only on an interface, implementation in infrastructure.

4 Cousins: hexagonal, onion, clean

Style	Vocabulary	Distinctive idea
Hexagonal (Cockburn, 2005)	ports and adapters	driving vs driven sides
Onion (Palermo, 2008)	rings: domain model, domain services, application services	layers around the domain model
Clean (Martin, 2012)	entities, use cases, adapters, frameworks	explicit use-case classes; the dependency rule

All three put the domain in the middle and push infrastructure to the edges. Pick the vocabulary your team understands; the boundaries matter more than the names.

WHEN IT'S OVERKILL

- Small CRUD services and prototypes: many files for little logic, each field mapped three times (request → command → entity → row).
- Short-lived code, or a framework you will never replace.
- Use it for complex, long-lived business rules; you can apply it to one complex module and keep the rest simply layered.

WORKED EXAMPLE: A FASTAPI INVOICING SERVICE

Routers are thin: they parse the request and call a use case such as `CreateInvoice`. The use case depends on an `InvoiceRepository` interface, not on SQLAlchemy. `SqlInvoiceRepository` lives in an `infrastructure` package. Unit tests call `CreateInvoice` directly with an `InMemoryInvoiceRepository` and never start FastAPI or a database. An import-linter rule (lesson 20) fails the build if `usecases` imports `fastapi` or `sqlalchemy`.

5 Key terms

Dependency rule: source dependencies point only inward

Use case: one application action, e.g. CreateInvoice

Entity: core business object and rules

Interface adapter: converts between use cases and the outside

Composition root: start-up code that wires implementations

DTO: simple data structure crossing a boundary

6 Interview questions

Q1. What is the dependency rule? BASIC

Source-code dependencies only point inward: entities and use cases never import frameworks, ORMs, databases or UI code.

Q2. Name the four rings. BASIC

Entities, use cases, interface adapters, and frameworks and drivers, from the centre outward.

Q3. How does a use case save data without depending on the database? SCENARIO

It defines a repository interface; the infrastructure ring implements it, and the composition root injects the implementation at start-up.

Q4. How do you keep the rule from eroding? SCENARIO

Enforce it in the build with import rules or architecture tests (import-linter, ArchUnit), and keep framework types out of use-case signatures.

Q5. When is clean architecture overkill? TRICKY

For small CRUD services and prototypes, where the extra layers and mappings cost more than they protect.

Q6. How is it different from hexagonal architecture? TRICKY

Barely: both keep the domain at the centre with dependencies pointing inward; clean architecture adds named rings and explicit use-case classes.

Next: SD-L-015 · Serverless backendroadmap.com